

CampusHub 校园论坛中期报告

个人阶段总结 - 姓名：杨尔灏 - 学号：24320319 - 2026 年 9 月

一、项目与个人负责方向

CampusHub 是一个面向校园场景的 Web 论坛项目，主要围绕帖子发布、帖子浏览、评论互动、个人中心、浏览记录和推荐排序等功能展开。本阶段我的工作重点集中在论坛的互动体验和帖子广场的信息筛选能力，包括点赞、收藏、评论点赞、热门排序、猜你喜欢推荐、tag 搜索与推荐算法优化。

团队虽然有前后端模块划分，但实际开发中采用按功能负责的方式。因此我负责的内容不是单纯前端或后端，而是围绕“互动与推荐”这条用户流程，同时完成后端接口、数据库关系、前端组件和页面联动。

二、个人完成内容概览

阶段	功能内容	主要代码文件	完成效果
D2	互动模块：帖子点赞、取消点赞、帖子收藏、取消收藏、评论点赞、取消评论点赞。	server/routes/interactions.js client/src/api/interactions.js client/src/components/InteractionButtons.vue	形成可复用互动组件，并通过接口操作 post_likes、favorites、comment_likes 等关系表。
D3	帖子广场推荐与排序：最新、热门、猜你喜欢；分类和 tag 结构调整；补充测试数据。	server/routes/posts.js client/src/views/PostsView.vue devdocs/sample_posts_seed.sql	支持热门排序和基于用户行为的 tag 偏好推荐；帖子 tag 可作为话题入口。
D4	推荐算法优化：加入时间衰退；修复详情页 tag 跳转；搜索支持 tag 模糊匹配；排序改为单选。	server/routes/posts.js client/src/views/PostDetailView.vue client/src/views/PostsView.vue	推荐结果更偏向近期兴趣；搜索和筛选状态更加一致，页面操作更清晰。

三、互动模块实现说明

互动模块的核心目标是记录“谁对什么内容产生了什么互动”。帖子点赞、帖子收藏和评论点赞都不是只修改一个数量字段，而是先在关系表中保存用户与内容的关系，再同步更新帖子或评论上的计数字段。

- 帖子点赞：使用 post_likes 保存 user_id 与 post_id 的对应关系。
- 帖子收藏：使用 favorites 保存 user_id 与 post_id 的对应关系。
- 评论点赞：使用 comment_likes 保存 user_id 与 comment_id 的对应关系。
- 前端组件：InteractionButtons.vue 封装点赞和收藏按钮，之后帖子列表和详情页都可以复用。

关键接口对应关系

功能	接口	说明
帖子点赞	POST /api/posts/:id/like	登录用户对帖子点赞，成功后返回 liked 与 likeCount。

功能	接口	说明
取消帖子点赞	DELETE /api/posts/:id/like	删除点赞关系，并使用 GREATEST 防止计数小于 0。
帖子收藏	POST /api/posts/:id/favorite	记录收藏关系，并更新 favorite_count。
取消收藏	DELETE /api/posts/:id/favorite	删除收藏关系，并返回最新收藏数。
评论点赞	POST /api/comments/:id/like	对评论建立点赞关系。
取消评论点赞	DELETE /api/comments/:id/like	删除评论点赞关系。

互动模块中比较重要的一点是事务处理。点赞或收藏不是一个单独动作，而是“插入/删除关系记录”和“更新计数字段”两个数据库操作。为了避免其中一步成功、另一步失败导致数据不一致，后端将相关 SQL 放在事务中执行。

```
INSERT INTO post_likes (user_id, post_id)
VALUES ($1, $2)
ON CONFLICT DO NOTHING;

UPDATE posts
SET like_count = like_count + 1
WHERE id = $2;
```

四、帖子广场推荐与热门排序

帖子广场是用户浏览内容的核心入口。本阶段我在 GET /api/posts 接口中加入 rank 参数，使后端可以根据不同排序策略返回帖子列表。目前前端已经调整为单选排序，用户可以在“最新、热门、猜你喜欢”三种方式中选择一种。

- 最新：主要依据帖子创建时间倒序排列。
- 热门：根据浏览数、点赞数、收藏数、评论数计算热度分。
- 猜你喜欢：根据用户浏览、点赞、收藏过的帖子标签计算兴趣权重。

热门排序公式

```
hotScore = view_count * 1 + like_count * 3 + favorite_count * 4 + comment_count * 5
```

这个公式中，评论和收藏权重较高，因为它们比单纯浏览更能说明用户对内容的兴趣；点赞次之；浏览权重最低，但仍可体现帖子曝光和阅读情况。

五、猜你喜欢推荐算法

猜你喜欢并不是固定推荐，而是从用户行为中提取标签偏好。后端会读取用户的 histories、post_likes 和 favorites，找到这些帖子绑定的 tag，然后按行为类型累加兴趣权重。

行为来源	基础权重	含义
浏览 histories	1	用户看过某类内容，说明有轻度兴趣。
点赞 post_likes	3	用户主动表达认可，兴趣强于浏览。
收藏 favorites	4	用户希望后续再次查看，兴趣最强。

为了解决新用户或没有行为记录的冷启动问题，如果用户还没有任何偏好数据，recommend 会暂时用热门分进行兜底排序，避免列表为空或排序完全无差异。

六、推荐时间衰退机制

后续测试时发现，用户很久以前的浏览、点赞、收藏如果一直保持同样权重，会让推荐结果不够贴近当前兴趣。因此我加入了时间衰退机制：越新的行为影响越大，越久以前的行为影响逐渐变小。

行为时间	保留权重
当天	100%
第 2 到第 5 天	70%
第 5 到第 14 天	40%
第 14 到第 30 天	20%
30 天以上	0%

代码中通过 decayWeight(timeExpression) 生成 SQL CASE 表达式，并分别作用到 histories.viewed_at、post_likes.created_at 和 favorites.created_at。这样同样是一次点赞或收藏，越接近当前时间，对推荐分的贡献越高。

七、tag 搜索与页面联动

tag 在本项目中被定位为帖子内部的话题关键词，不再作为帖子广场顶部固定展示的一排筛选项。用户发帖时可以选择或填写 tag；其他用户浏览帖子时，可以点击帖子中的 tag 进入相关内容搜索。

- 帖子广场内点击 tag：调用 `searchByTag(tag)`，设置 `keyword` 和 `tag` 后重新加载列表。
- 帖子详情页点击 tag：通过 `router.push` 跳转到 `PostPage`，并在 URL query 中携带 `tag` 和 `keyword`。
- 帖子广场读取 query：页面 `mounted` 和 `route.query` 变化时读取 `tag/keyword`，实现从详情页返回后的自动搜索。
- 搜索状态修复：如果用户手动删除搜索框内容，或把内容改成普通关键词，会自动清空内部 `tag` 筛选状态。

另外，我将普通搜索范围从标题和正文扩展到了 tag。现在用户在搜索框直接输入“经验分享”时，即使帖子标题和正文没有出现该词，只要帖子绑定了这个 tag，也可以被搜索出来。

```
p.title ILIKE $keyword
OR p.content ILIKE $keyword
OR EXISTS (
  SELECT 1 FROM post_tags pt
  JOIN tags t ON t.id = pt.tag_id
  WHERE pt.post_id = p.id AND t.name ILIKE $keyword
)
```

八、关键问题与解决过程

1. 重复点赞与重复收藏

问题：点赞和收藏功能如果只靠前端按钮限制，用户重复请求时仍可能造成重复记录和计数错误。

解决：在 `post_likes`、`favorites`、`comment_likes` 表中使用 `user_id + 内容 id` 的唯一约束，并在插入时使用 `ON CONFLICT DO NOTHING`，保证同一用户对同一对象只能建立一条互动关系。

2. 计数字段与关系表一致性

问题：互动关系表记录了谁点赞或收藏，但帖子列表显示的是 `like_count`、`favorite_count` 等计数字段。如果两者更新不一致，页面数字就会失真。解决：将关系表操作和计数字段更新放进同一个事务，失败时统一回滚。

3. 合并 develop 后出现冲突

问题：队友在 `develop` 中新增帖子详情页、附件上传和评论功能，我也修改了 `PostsView.vue`、`posts.js` 等文件，合并时出现冲突。解决：保留队友负责的详情页和附件能力，同时重新合入自己的推荐排序、分类 tag、搜索逻辑，保证两个功能能够共存。

4. 详情页点击 tag 后筛选状态残留

问题：从详情页点击 tag 回到帖子广场后，如果用户只是手动删除搜索框文字，内部 `tag` 条件仍然存在，导致页面看起来清空了搜索但结果仍被过滤。解决：在输入事件中检测 `keyword` 和 `tag` 是否一致，如果为空或变成普通关键词，就清除 `tag` 状态。

5. 本地数据库缺少新增表导致 500

问题：帖子详情页接口返回 500，最初以为是评论数据缺失。查看后端后发现详情接口会查询 post_attachments 表，而本地 PostgreSQL 还没有同步该表。解决：补建 post_attachments 表和索引，不重新运行整份 DROP 脚本，避免清空已有数据。

九、阶段成果与后续计划

目前，我负责的互动和推荐相关功能已经形成较完整闭环：用户可以在帖子列表和详情页进行点赞、收藏，进入详情页会记录浏览行为，帖子广场可以根据最新、热门或猜你喜欢进行排序，搜索框可以同时匹配标题、正文和 tag，详情页 tag 也可以跳回帖子广场搜索。

后续计划主要包括：继续测试不同账号下推荐结果的差异；将 feature/recommendation-decay 分支合并到 develop；补充推荐算法说明文档；如果时间允许，增加举报或内容审核模块，提高校园社区项目的完整性和内容安全性。